TSC — Arquitectura Software

> **Documento**: ARCHITECTURE_SW.md **Documento principal**: ARCHITECTURE.md — Mantener los 3 documentos sincronizados **Documento HW**: ARCHITECTURE_HW.md

1. Diagrama del Software

```
flowchart TD
    subgraph BACKEND["Backend Python"]
        MR["Modbus Reader\n(pymodbus)"]
        OT["OT Manager\n(cronómetro inteligente)"]
        AL["Alarm Engine\n(detección anomalías)"]
    end

    subgraph DB["Base de Datos (SQLite)"]
        R["Registro térmico (cada 30s)"]
        H["Historial cargas y OTs"]
        E["Eventos y alarmas"]
    end

    subgraph FRONT["Frontend Web (Dashboard)"]
        CV["Curvas temperatura tiempo real"]
        CA["Estado cargas activas"]
        HI["Histórico por fecha/carga/OT"]
    end

    MR --> R
    OT --> H
    AL --> E
    R --> CV
    H --> CA
    H --> HI
```

2. Funcionalidades — FASE 1

2.1 Registro Térmico (SURSULF)

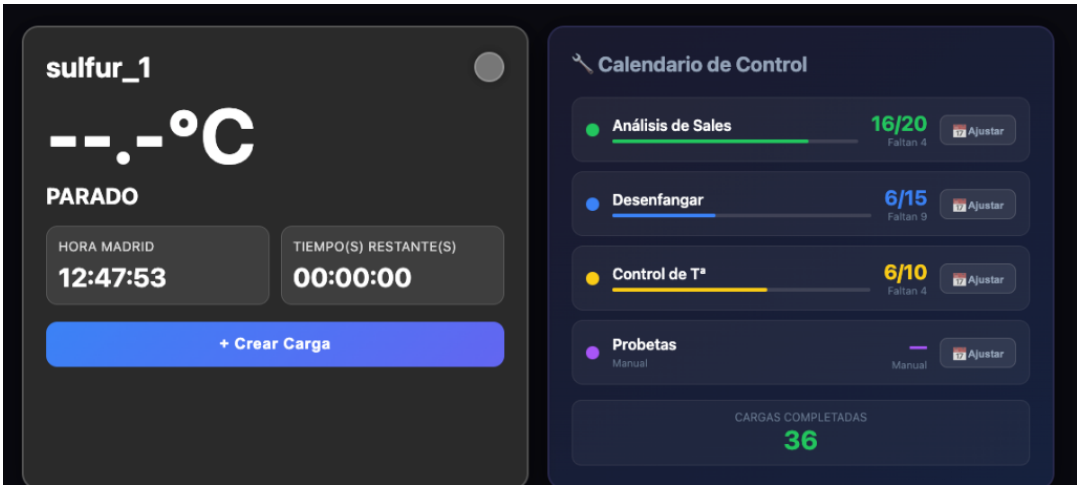
- Lectura cada 30 segundos de todos los hornos conectados
- Almacenamiento inmediato en SQLite local
- Visualización de curvas en tiempo real en el dashboard
- Rangos: 0-800°C (configurables por horno)

2.2 Gestión de Cargas y Subcargas

- Botón **+ Crear Carga** en el panel de Control
- Soporte multi-subcarga: cada subcarga con su OT, peso (kg) y duración
- OT editable manualmente por operarios (ej: `11/2026`)
- Al pulsar **START**: se guardan todas las subcargas en SQLite + Supabase (dual-write)
- Cada subcarga en Supabase = una fila: `Carga N DD/MM/YY [furnace] Sub X/Y`
- Popup de confirmación al llegar countdown a 0: *"¿Retirar la Carga de X min (OT X)?"*
- Al confirmar: se graba `end_time` , `duration_s` , `total_minutes`

2.3 Calendario de Control (Mantenimiento)

- Tareas: Análisis de Sales (20), Desenfangar (15), Control de Tª (10), Probetas (manual)
- Botón 📅 **Ajustar** por tarea: abre selector de fecha "¿Cuándo fue el último control?"
- Solo cuenta días con cargas reales (excluye fines de semana y festivos sin actividad)
- Muestra: Último control, Próximo estimado, barra de progreso
- Sincronizado entre Control y Dashboard



Menú Control — Pre-producción 28/03/2026

2.4 Alarmas Críticas

- **Fallo termopar**: detección automática si la temperatura cae bruscamente
- **Sobrecalentamiento**: umbral configurable por horno
- **Timer excedido**: alerta si una OT supera el tiempo requerido
- **Canales**: Email (Resend), WhatsApp (Fase 2), sirena HW (Fase 2)

2.5 Visualización e Histórico

- **Control**: `https://tsc.tail8dce43.ts.net/` — panel principal con cargas, timers y mantenimiento
- **Dashboard**: `https://tsc.tail8dce43.ts.net/dashboard` — gráficos, calendario, barras de duración

- **Monitor**: `https://tsc.tail8dce43.ts.net/monitor` — vista operador con calendario de control
- **DB Viewer**: `https://tsc.tail8dce43.ts.net/db` — tablas de cargas, eventos, OTs, trazabilidad
- Hover en barras del Dashboard muestra: `Carga N DD/MM/YY – OT X – X min`
- Filtrado por fecha, horno, cliente, OT
- Exportación CSV/JSON

3. Base de Datos

3.1 Esquema SQLite

```sql
-- Lecturas térmicas (controlado por Temp Tracker ON/OFF)
CREATE TABLE readings (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp   DATETIME DEFAULT CURRENT_TIMESTAMP,
    furnace     TEXT NOT NULL DEFAULT 'sulfur_1',
    slave_id    INTEGER,
    temp_sales  REAL,
    temp_cameras REAL,
    set_point   REAL,
    raw_value   INTEGER,
    status      TEXT
);

-- Cargas diarias
CREATE TABLE loads (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    name          TEXT NOT NULL,
    furnace       TEXT NOT NULL,
    date          DATE DEFAULT CURRENT_DATE,
    start_time    DATETIME,
    end_time      DATETIME,
    duration_s    INTEGER,
    total_minutes REAL,
    ot_number     TEXT,
    duration_min  REAL,
    client_id     TEXT,
    pieces_id     TEXT,
    status        TEXT DEFAULT 'pending'
);

-- Subcargas / Órdenes de Trabajo (OT)
CREATE TABLE work_orders (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    load_id      INTEGER REFERENCES loads(id),
    ot_number    TEXT NOT NULL,
    client_id    TEXT,
    client_name  TEXT,
    piece_id     TEXT,
    piece_ref    TEXT,
    extra_id_1   TEXT,
    extra_id_2   TEXT,
    extra_id_3   TEXT,
    required_min REAL NOT NULL DEFAULT 30,
    start_time   DATETIME,
    end_time     DATETIME,
    status       TEXT DEFAULT 'pending',
    process      TEXT DEFAULT 'SURSULF',
    weight       TEXT,
    reference    TEXT,
    done         INTEGER DEFAULT 0,
    done_time    DATETIME,
    duration_min REAL,
    piece_type   TEXT,
    material     TEXT,
    industry     TEXT,
    target_temp  REAL,
    hold_time_min INTEGER,
```

```
    cooling_method TEXT,
    quality_check TEXT DEFAULT 'pending',
    quality_notes TEXT,
    batch_id     TEXT
);

-- Eventos
CREATE TABLE events (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp     DATETIME DEFAULT CURRENT_TIMESTAMP,
    event_type    TEXT NOT NULL,
    furnace       TEXT,
    details       TEXT,
    email_sent    INTEGER DEFAULT 0,
    whatsapp_sent INTEGER DEFAULT 0
);

-- Alarmas
CREATE TABLE alarms (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp    DATETIME DEFAULT CURRENT_TIMESTAMP,
    type         TEXT NOT NULL,
    severity     TEXT DEFAULT 'warning',
    message      TEXT,
    acknowledged BOOLEAN DEFAULT FALSE
);

-- Temperature Tracking (snapshots enriquecidos con contexto OT)
CREATE TABLE temperature_tracking (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp       DATETIME NOT NULL,
    furnace         TEXT NOT NULL,
    temperature     REAL,
    ot_number       TEXT,
    subload_summary TEXT,
    load_status     TEXT,
    date            DATE NOT NULL
);

-- Búsquedas LLM (historial de consultas IA)
CREATE TABLE llm_searches (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp    DATETIME DEFAULT CURRENT_TIMESTAMP,
    question     TEXT NOT NULL,
    sql_generated TEXT,
    engine       TEXT,       -- 'local' o 'ollama'
    result_count INTEGER DEFAULT 0,
    error        TEXT,
    date         DATE NOT NULL
);

-- Índices
CREATE INDEX idx_readings_ts ON readings(timestamp);
CREATE INDEX idx_readings_furnace ON readings(furnace);
CREATE INDEX idx_loads_date ON loads(date);
CREATE INDEX idx_events_ts ON events(timestamp);
CREATE INDEX idx_temp_tracking_date ON temperature_tracking(date);
```

3.2 Schema Supabase PostgreSQL — Recrear desde cero

> **Para reconstruir Supabase desde cero**, ejecutar este SQL en el SQL Editor. Refleja exactamente el estado de producción a 27/03/2026.

```
-- ============================================================
-- TSC SUPABASE — Schema completo PostgreSQL
-- Proyecto: httfohqoikkptywupkrs
-- Para recrear: Supabase SQL Editor → pegar todo → Run
-- ============================================================

CREATE EXTENSION IF NOT EXISTS pgaudit;

CREATE TABLE IF NOT EXISTS loads (
    id            SERIAL PRIMARY KEY,
    name          TEXT NOT NULL,
    furnace       TEXT DEFAULT 'sulfur_1',
    date          DATE DEFAULT CURRENT_DATE,
    start_time    TIMESTAMPTZ,
    end_time      TIMESTAMPTZ,
    duration_s    INTEGER,
    total_minutes REAL,
    duration_min  REAL,
    ot_number     TEXT,
    client_id     TEXT,
    pieces_id     TEXT,
    notes         TEXT,
    status        TEXT DEFAULT 'pending'
        CHECK (status IN ('pending', 'active', 'completed'))
);

CREATE TABLE IF NOT EXISTS readings (
    id           SERIAL PRIMARY KEY,
    timestamp    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    furnace      TEXT DEFAULT 'sulfur_1',
    temp_sales   REAL,
    temp_cameras REAL,
    set_point    REAL,
    load_id      INTEGER REFERENCES loads(id)
);

CREATE TABLE IF NOT EXISTS work_orders (
    id            SERIAL PRIMARY KEY,
    load_id       INTEGER REFERENCES loads(id),
    ot_number     TEXT NOT NULL,
    client_id     TEXT,
    client_name   TEXT,
    piece_id      TEXT,
    piece_ref     TEXT,
    extra_id_1    TEXT,
    extra_id_2    TEXT,
    extra_id_3    TEXT,
    required_min  REAL NOT NULL DEFAULT 30,
    start_time    TIMESTAMPTZ,
    end_time      TIMESTAMPTZ,
    status        TEXT DEFAULT 'pending',
    process       TEXT DEFAULT 'SURSULF',
    weight        TEXT,
    reference     TEXT,
    done          BOOLEAN DEFAULT FALSE,
    done_time     TIMESTAMPTZ,
    duration_min  REAL,
```

```
    piece_type    TEXT,
    material      TEXT,
    industry      TEXT,
    target_temp   REAL,
    hold_time_min INTEGER,
    cooling_method TEXT,
    quality_check TEXT DEFAULT 'pending',
    quality_notes TEXT,
    batch_id      TEXT
);

CREATE TABLE IF NOT EXISTS events (
    id            SERIAL PRIMARY KEY,
    timestamp     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    event_type    TEXT NOT NULL,
    furnace       TEXT,
    details       TEXT,
    email_sent    BOOLEAN DEFAULT FALSE,
    whatsapp_sent BOOLEAN DEFAULT FALSE
);

CREATE TABLE IF NOT EXISTS alarms (
    id           SERIAL PRIMARY KEY,
    timestamp    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    type         TEXT NOT NULL,
    severity     TEXT DEFAULT 'warning'
        CHECK (severity IN ('info', 'warning', 'critical')),
    message      TEXT,
    acknowledged BOOLEAN DEFAULT FALSE
);

CREATE TABLE IF NOT EXISTS temperature_tracking (
    id              SERIAL PRIMARY KEY,
    timestamp       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    furnace         TEXT NOT NULL,
    temperature     REAL,
    ot_number       TEXT,
    subload_summary TEXT,
    load_status     TEXT,
    date            DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE IF NOT EXISTS llm_searches (
    id           BIGSERIAL PRIMARY KEY,
    timestamp    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    question     TEXT NOT NULL,
    sql_generated TEXT,
    engine       TEXT,
    result_count INTEGER DEFAULT 0,
    error        TEXT,
    date         TEXT NOT NULL
);

-- Índices
CREATE INDEX IF NOT EXISTS idx_readings_timestamp ON readings(timestamp);
CREATE INDEX IF NOT EXISTS idx_readings_furnace ON readings(furnace);
CREATE INDEX IF NOT EXISTS idx_loads_date ON loads(date);
CREATE INDEX IF NOT EXISTS idx_loads_status ON loads(status);
```

```
CREATE INDEX IF NOT EXISTS idx_work_orders_ot ON work_orders(ot_number);
CREATE INDEX IF NOT EXISTS idx_work_orders_load ON work_orders(load_id);
CREATE INDEX IF NOT EXISTS idx_events_timestamp ON events(timestamp);
CREATE INDEX IF NOT EXISTS idx_events_type ON events(event_type);
CREATE INDEX IF NOT EXISTS idx_alarms_timestamp ON alarms(timestamp);
CREATE INDEX IF NOT EXISTS idx_temp_tracking_date ON temperature_tracking(date);
CREATE INDEX IF NOT EXISTS idx_temp_tracking_ts ON temperature_tracking(timestamp);
CREATE INDEX IF NOT EXISTS idx_llm_requests_ts ON llm_requests(timestamp);
CREATE INDEX IF NOT EXISTS idx_llm_requests_user ON llm_requests(user_id);
```

3.3 Decisión técnica — Comparativa

DB	Auditoría	Escalabilidad	Complejidad	Coste	Ideal pa
SQLite	⚠️ Archivo editable, sin audit log	~1M filas	Cero config	Gratis	Fase 1 / Demo
PostgreSQL	✅ `pgaudit`, triggers inmutables	Millones	Servidor necesario	Gratis (Supabase/ Neon)	Fase 2 / Producci
TimescaleDB	✅ (es PostgreSQL)	Optimizado IoT	Extensión extra	Gratis (25GB Cloud)	Fase 3 / Alto volumen

3.4 Estrategia por fases

Fase	Base de datos	Motivo
Fase 1 (ahora)	SQLite	Rápido, local, demo inmediata
Fase 2 (producción)	PostgreSQL (Supabase)	Auditable, cloud, panel web
Fase 3 (alto volumen)	TimescaleDB	IoT optimizado, sobre PostgreSQL

> La migración SQLite → PostgreSQL es trivial: mismas queries SQL, solo cambia la conexión.

4. Notificaciones

4.1 Email (Resend)

Se usa la API HTTP de Resend para enviar emails al hacer START/STOP.

Evento	Asunto email	Destinatarios
START carga	`TSC INICIO CARGA - SULFUR_1`	alebronlobo81@gmail.com, d.serrano.9080@gmail.com
STOP carga	`TSC FIN - SULFUR_1 (45m 30s)`	alebronlobo81@gmail.com, d.serrano.9080@gmail.com

Config: `/home/tsc/furnace_monitor/notifier.py` usa `RESEND_API_KEY` vía HTTP POST a `api.resend.com/emails`.

4.2 WhatsApp (Fase 2)

- Ya tenemos infraestructura WhatsApp en el VPS (oc_control)
- Integración via API para alertas de alarma y fin de carga

5. Variables de entorno (`.env`)

```
# GitHub
GITHUB_TOKEN=ghp_...
GITHUB_USER=nexusfinlabs

# WiFi
WIFI_SSID=Livebox6-9D67
WIFI_PASS=tcQ5x3eYoaed

# Modbus
MODBUS_PORT=/dev/ttyUSB0    # Linux
# MODBUS_PORT=/dev/cu.usbserial-XXXX  # macOS
MODBUS_BAUD=9600
MODBUS_SLAVE_ID=1

# Dashboard
DASHBOARD_PORT=8080
DASHBOARD_HOST=0.0.0.0

# Tailscale
TAILSCALE_HOSTNAME=tsc-planta

# Resend (Email API)
RESEND_API_KEY=re_PJ4SJgLF_KJXVutpAfyoEWsjdpB8ncJfg
```

6. Arranque rápido

```
# Opción 1: Script automático (recomendado)
./start.sh

# Opción 2: Manual
python3 scripts/modbus_reader.py &
python3 scripts/dashboard.py
# → Abre http://localhost:8080
```

6.1 Acceder al Dashboard

Desde	URL
PC de planta	`http://localhost:8080`
Misma red	`http://192.168.1.X:8080`
Remoto (Tailscale)	`http://100.104.65.25:8080`

6.2 Comandos de desarrollo

```
# Lanzar en modo desarrollo (con hot-reload)
python3 scripts/dashboard.py --debug

# Simular lecturas Modbus (sin hardware)
python3 scripts/modbus_reader.py --simulate

# Ver logs
tail -f logs/temperature.log
tail -f logs/alarms.log

# Exportar histórico
python3 scripts/export_history.py --from 2026-03-01 --to 2026-03-18 --format csv
```

7. Plan de Ejecución (Fase 1)

Total: 160 horas · **Presupuesto**: 3.800€ + IVA

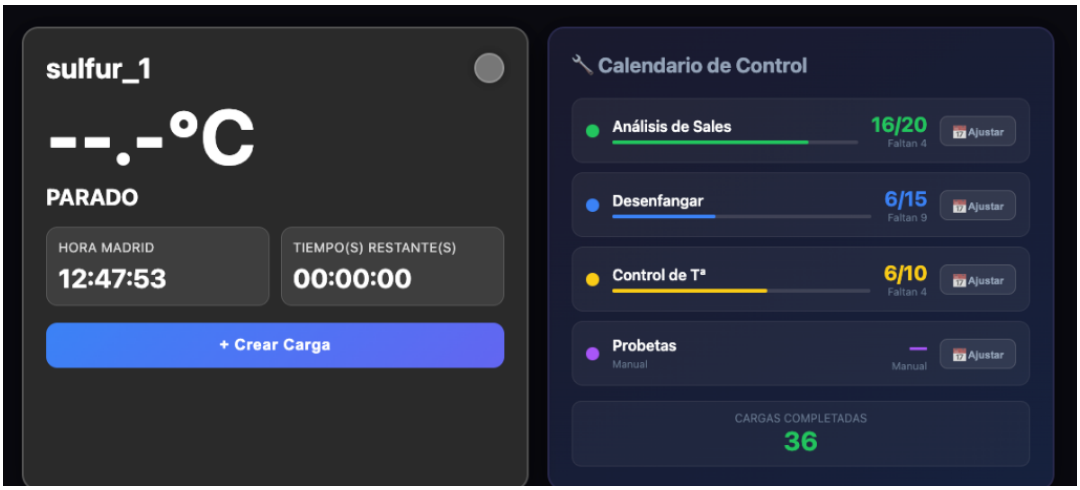
#	Sección	Fecha	Lugar	Estado
1	✅ **Verificación Pre-producción** — Conexión HW, TASI, cargas reales	Viernes 21 marzo	Planta TSC, 10:00–17:00	✅ Completado
2	✅ **Verificación Aplicación** — Menús Control / Monitor / Dashboard / Base de Datos	Viernes 27 marzo	Online	✅ Completado
3	🚀 **Paso a Producción** — Conexión TASI físico + instalación drivers termopares + operación real	Lunes 30 marzo	Planta TSC	⬜ Pendiente

✅ Verificación pre-producción — Viernes 21 de marzo (en Planta TSC)

Hora	Tarea	Responsable	Estado
10:00	Conectar TASI TA612C al Mini PC en planta	Alberto + Darío	✅
10:30	Verificar lectura `/dev/ttyUSB1` protocolo AA 55	Alberto	✅
11:00	Comprobar temperaturas Sales + Cámaras en dashboard	Alberto + Darío	✅
11:30	Crear primera carga real desde la pantalla de planta	Darío (operario)	✅
12:00	Verificar dual-write: SQLite local + Supabase cloud	Alberto	✅
12:30	Comprobar email START/STOP (Resend → Gmail)	Alberto	✅
13:00	🍽️ Pausa almuerzo	—	—
14:00	Revisar histórico y trazabilidad en `/db`	Darío	✅
14:30	Probar acceso remoto vía Tailscale desde Mac Alberto	Alberto	✅
15:00	Ajuste de SP y thresholds	Alberto	✅

Hora	Tarea	Responsable	Estado
	según temperaturas reales		
15:30	Documentar incidencias y pendientes de Fase 2	Alberto	✅
16:00	Prueba de carga completa supervisada	Darío + Alberto	✅
16:30	Cierre y acta de verificación	Alberto + Darío	✅
17:00	**FIN de jornada**	—	—

✅ Verificación Aplicación — Menús Control / Monitor / Dashboard / Base de Datos · Viernes 27 de marzo (online)



Menu Control TSC en producción

Verificación completa de los 4 menús de la aplicación con el sistema en producción. La interfaz refleja el estado real del horno `sulfur_1` y el Calendario de Control con las 4 categorías activas:

- 🟢 **Análisis de Sales** — 6/20 completadas (Faltan 14)
- 🔵 **Desenfangar** — 11/15 completadas (Faltan 4)
- 🟡 **Control de Tª** — 6/10 completadas (Faltan 4)
- 🟣 **Probetas** — Manual (sin contador automático)

Menús verificados el 27/03/2026:

Menú	Verificado	Notas
Control	✅	Crear carga, sirena, calendario, hora Madrid, cronómetro
Monitor	✅	Vista de gran formato operativa en nave
Dashboard	✅	Gráficas, histórico y comparativas funcionando
Base de Datos	✅	Trazabilidad, OTs, dual-write SQLite + Supabase

Estado de componentes a 27/03/2026:

Componente	Estado	Notas
TASI TA612C	🟡 Desconectado	Pendiente de conexión física estable
Dashboard web `:5000`	✅ Operativo	Accesible en red local y Tailscale
Dual-write Supabase	✅ Activo	Todas las cargas se sincronizan
Email START/STOP	✅ Activo	Resend → Gmail
Calendario de Control	✅ Activo	26 cargas completadas
PerfectPrime TC0378	⬜ Pendiente	Integración programada Fase 2
UNI-T UT325F	⬜ Pendiente	Integración programada Fase 2

🚀 Paso a Producción — Lunes 30 de marzo

Arranque definitivo del sistema en planta TSC con conexión física del TASI TA612C y operación supervisada por Darío.

#	Tarea	Responsable	Estado
1	Conectar TASI TA612C físicamente al Mini PC	Darío + Alberto (remoto)	☐
2	Verificar lectura en vivo de Sales + Cámaras	Alberto (Tailscale)	☐
3	Primera carga real del lunes con TASI activo	Darío (operario)	☐
4	Confirmar dual-write con lectura Supabase cloud	Alberto	☐
5	Sistema autónomo — operario opera sin supervisión	Darío	☐

> [!IMPORTANT] A partir del **lunes 30 de marzo**, el sistema opera en producción real. Cualquier incidencia → WhatsApp Alberto.

8. FASE 2 — PostgreSQL + Inteligencia Artificial

8.1 PostgreSQL en la Nube (Supabase)

```
flowchart TD
    A["Operario pulsa START"] --> B["app.py recibe POST"]
    B --> C["1. SQLite local (instantáneo)"]
    B --> D["2. PostgreSQL Supabase (async)"]
    B --> E["3. Resend email (async)"]
    C -. "backup offline" .-> F["Funciona sin internet"]
    D -. "nube" .-> G["Panel web Supabase"]
    E -. "email" .-> H["Alberto + Darío"]
```

> Si internet cae, SQLite sigue funcionando. Cuando vuelva, se sincronizan los registros pendientes.

PostgreSQL gratis — opciones de hosting:

Servicio	Plan gratis	Límite	URL
Supabase	Sí	500MB, 2 proyectos	supabase.com
Neon	Sí	512MB, branching	neon.tech
Railway	Sí (trial)	1GB, $5/mes después	railway.app
Self-hosted (VPS)	Ya lo tenemos	Sin límite	openclawd-vps

8.1.1 PostgreSQL local (Docker) — ✅ Implementado

Para desarrollo y testing local usamos Docker Compose con PostgreSQL 16 + `pgaudit` :

```
# 1. Abrir Docker Desktop (macOS) o asegurar que Docker está corriendo
# 2. Arrancar PostgreSQL:
cd ~/Desktop/SW_AI/tsc && docker compose up -d

# 3. Verificar que arrancó:
docker compose ps       # → Estado: healthy
docker compose logs -f  # → Ver logs en tiempo real

# 4. Conectar con psql:
psql -h localhost -p 5432 -U tsc -d tsc_furnace
# Password: tsc_secure_2026

# 5. Parar:
docker compose down      # Preserva datos (volumen pgdata)
docker compose down -v   # ⚠️ BORRA DATOS (reinicio limpio)
```

Archivos: - `docker-compose.yml` — PostgreSQL 16 con pgaudit preloaded - `db/pgaudit.conf` — Configuración de auditoría - `db/init/001_schema.sql` — Esquema principal (5 tablas) - `db/init/002_audit_trail.sql` — Audit trail + triggers inmutables

8.1.2 pgaudit — Auditoría a nivel de servidor

`pgaudit` registra en los logs de PostgreSQL **toda operación de escritura y DDL**, incluyendo: - Quién ejecutó la query (usuario) - Qué query se ejecutó (SQL completo) - Cuándo se ejecutó (timestamp)

```
# Ver logs de auditoría de pgaudit:
docker compose logs postgres | grep AUDIT
```

Configuración (`db/pgaudit.conf`):

```
pgaudit.log = 'write, ddl'     ← INSERT, UPDATE, DELETE + CREATE, ALTER, DROP
pgaudit.log_catalog = off       ← No loguear metadatos del sistema
pgaudit.log_level = log         ← Nivel de log
```

8.1.3 Audit Trail — Tabla `audit_log`

Además de `pgaudit` (logs del servidor), cada cambio se registra en una **tabla consultable** (`audit_log`) con datos before/after en JSON:

```
-- Ver todos los cambios de hoy
SELECT * FROM audit_log WHERE DATE(timestamp) = CURRENT_DATE ORDER BY timestamp DESC

-- Ver intentos bloqueados (alguien intentó borrar o modificar datos protegidos)
SELECT * FROM v_intentos_bloqueados;

-- Resumen diario de actividad
SELECT * FROM v_audit_diario;

-- ¿Quién modificó la OT-2026-0042?
SELECT * FROM audit_log
WHERE table_name = 'work_orders'
  AND new_data->>'ot_number' = 'OT-2026-0042'
ORDER BY timestamp;
```

8.1.4 Triggers inmutables — Datos protegidos

Tabla	Protección	Qué pasa si se intenta
`readings`	🚫 No DELETE	Error + registro en audit_log
`readings`	🔒 `timestamp` inmutable	Error + registro en audit_log
`readings`	🔒 `temp_sales` inmutable	Error + registro en audit_log
`readings`	🔒 `temp_cameras` inmutable	Error + registro en audit_log
`events`	🚫 No DELETE	Error + registro en audit_log
`events`	🔒 `timestamp` inmutable	Error + registro en audit_log
`alarms`	🚫 No DELETE	Error + registro en audit_log
`alarms`	🔒 `timestamp` inmutable	Error + registro en audit_log
`audit_log`	🛡️ No UPDATE ni DELETE	La propia tabla de auditoría es inmutable

> ⚠️ **Se permiten** updates de campos operativos (ej: `set_point` , `load_id` en readings, `status` en loads). Solo se bloquean los datos de medición que deben ser inmutables para normativa ISO.

8.1.5 Troubleshooting PostgreSQL

```
# ── Arranque y conectividad ──────────────────────────────
# PostgreSQL no arranca
docker compose down && docker compose up -d
docker compose logs postgres | tail -30

# Error "port 5432 already in use"
lsof -i :5432                    # Ver qué proceso usa el puerto
kill $(lsof -t -i:5432)          # Matar proceso
docker compose up -d

# Error de autenticación
psql -h localhost -p 5432 -U tsc -d tsc_furnace
# Password: tsc_secure_2026

# ═══════════════════════════════════════════════════════════
# QUERIES ÚTILES — TSC PostgreSQL (tsc_furnace)
# ═══════════════════════════════════════════════════════════

# ── 1. LECTURAS DE TEMPERATURA ───────────────────────────

# Últimas 10 lecturas de cualquier horno
SELECT * FROM readings ORDER BY timestamp DESC LIMIT 10;

# Lecturas de un horno específico en las últimas 24h
SELECT timestamp, temp_sales, temp_cameras, set_point
FROM readings
WHERE furnace = 'sulfur_1'
  AND timestamp > NOW() - INTERVAL '24 hours'
ORDER BY timestamp;

# Lecturas vinculadas a una carga específica
SELECT r.timestamp, r.temp_sales, r.temp_cameras, l.name AS carga
FROM readings r
JOIN loads l ON r.load_id = l.id
WHERE l.id = 5
ORDER BY r.timestamp;

# Temperatura promedio por horno (últimas 24h)
SELECT furnace,
       ROUND(AVG(temp_sales)::numeric, 1)   AS avg_sales,
       ROUND(AVG(temp_cameras)::numeric, 1) AS avg_cameras,
       ROUND(MAX(temp_sales)::numeric, 1)   AS max_sales
FROM readings
WHERE timestamp > NOW() - INTERVAL '24 hours'
GROUP BY furnace;

# ── 2. CARGAS ────────────────────────────────────────────

# Cargas activas ahora mismo (con sus OTs)
SELECT l.id, l.name, l.furnace, l.start_time,
       wo.ot_number, wo.client_name, wo.required_min
FROM loads l
LEFT JOIN work_orders wo ON wo.load_id = l.id
WHERE l.status = 'active'
ORDER BY l.start_time;

# Todas las cargas de hoy
```

```
SELECT id, name, furnace, status,
       start_time, end_time, total_minutes
FROM loads
WHERE DATE(start_time) = CURRENT_DATE
ORDER BY start_time;

# Cargas completadas esta semana con duración
SELECT name, furnace,
       start_time, end_time,
       ROUND(total_minutes::numeric, 1) AS duracion_min
FROM loads
WHERE status = 'completed'
  AND start_time > NOW() - INTERVAL '7 days'
ORDER BY start_time DESC;

# ── 3. ÓRDENES DE TRABAJO ──────────────────────────────

# Todas las OTs de un cliente
SELECT ot_number, client_name, piece_ref,
       required_min, status, process
FROM work_orders
WHERE client_name ILIKE '%comesa%'
ORDER BY id DESC;

# OTs pendientes con tiempo requerido > 120 min
SELECT wo.ot_number, wo.client_name, wo.required_min,
       l.name AS carga, l.furnace
FROM work_orders wo
JOIN loads l ON wo.load_id = l.id
WHERE wo.status = 'pending'
  AND wo.required_min > 120
ORDER BY wo.required_min DESC;

# ── 4. EVENTOS ─────────────────────────────────────────

# Todos los eventos de hoy (START/STOP/ALARM)
SELECT timestamp, event_type, furnace,
       email_sent, whatsapp_sent, details
FROM events
WHERE DATE(timestamp) = CURRENT_DATE
ORDER BY timestamp DESC;

# Eventos con email enviado pero WhatsApp pendiente
SELECT * FROM events
WHERE email_sent = 1 AND whatsapp_sent = 0
ORDER BY timestamp DESC;

# ── 5. ALARMAS ─────────────────────────────────────────

# Alarmas no reconocidas (activas)
SELECT * FROM alarms
WHERE acknowledged = FALSE
ORDER BY timestamp DESC;

# Alarmas críticas de los últimos 7 días
SELECT timestamp, type, message
FROM alarms
WHERE severity = 'critical'
```

```
  AND timestamp > NOW() - INTERVAL '7 days'
ORDER BY timestamp DESC;

# ── 6. CONSULTAS LLM ──────────────────────────────────────

# Últimas 20 consultas al LLM
SELECT timestamp, user_id, channel, model,
       LEFT(prompt, 80) AS prompt_preview,
       tokens_in, tokens_out, duration_ms
FROM llm_requests
ORDER BY timestamp DESC
LIMIT 20;

# Coste estimado por usuario (GPT-4o-mini: $0.15/1M input tokens)
SELECT user_id,
       COUNT(*) AS total_consultas,
       SUM(tokens_in) AS total_tokens_in,
       ROUND((SUM(tokens_in) / 1000000.0 * 0.15)::numeric, 4) AS coste_usd
FROM llm_requests
GROUP BY user_id
ORDER BY total_tokens_in DESC;

# ── 7. AUDIT TRAIL ────────────────────────────────────────

# Ver todos los cambios de hoy
SELECT * FROM audit_log
WHERE DATE(timestamp) = CURRENT_DATE
ORDER BY timestamp DESC;

# Intentos bloqueados (alguien intentó borrar datos)
SELECT * FROM v_intentos_bloqueados;

# Resumen diario de actividad
SELECT * FROM v_audit_diario;

# ── 8. MANTENIMIENTO ──────────────────────────────────────

# Tamaño de la base de datos
SELECT pg_size_pretty(pg_database_size('tsc_furnace'));

# Número de filas por tabla
SELECT tabla, count FROM (
  SELECT 'readings'     AS tabla, COUNT(*) FROM readings
  UNION ALL SELECT 'loads',        COUNT(*) FROM loads
  UNION ALL SELECT 'work_orders',  COUNT(*) FROM work_orders
  UNION ALL SELECT 'events',       COUNT(*) FROM events
  UNION ALL SELECT 'alarms',       COUNT(*) FROM alarms
  UNION ALL SELECT 'llm_requests', COUNT(*) FROM llm_requests
  UNION ALL SELECT 'audit_log',    COUNT(*) FROM audit_log
) t ORDER BY tabla;

# Backup manual
docker compose exec postgres pg_dump -U tsc tsc_furnace > backup_$(date +%Y%m%d).sql

# Restaurar backup
cat backup_20260319.sql | docker compose exec -T postgres psql -U tsc -d tsc_furnace
```

8.1.6 Setup local (sin Docker — Homebrew macOS)

```
# Si ya tienes PostgreSQL via Homebrew:
brew services start postgresql@17
createdb tsc_furnace

# Ejecutar migrations (saltar pgaudit que no está en Homebrew):
grep -v "CREATE EXTENSION" db/init/001_schema.sql | psql -d tsc_furnace
grep -v "CREATE EXTENSION" db/init/002_audit_trail.sql | psql -d tsc_furnace

# Verificar
psql -d tsc_furnace -c "\dt"
```

8.1.7 ¿Dónde veo los datos de PostgreSQL?

Opción	URL / Comando	Cuándo
Terminal (psql)	`psql -d tsc_furnace`	Siempre disponible
pgAdmin (web GUI)	`brew install --cask pgadmin4` → `localhost:5050`	Desarrollo local
Supabase Studio	`app.supabase.com` → tu proyecto	✅ **Ya configurado**
Dashboard TSC	`http://100.104.65.25:8080/db`	✅ **Funcionando**
Grafana	`http://localhost:3000`	✅ **Configurado**

> ✅ **AHORA**: el viewer en `:8080/db` muestra los datos de SQLite. Los START/STOP se guardan en ambas bases de datos (SQLite local + Supabase cloud) automáticamente via dual-write.

8.1.8 Roadmap PostgreSQL → Producción

#	Tarea	Dificultad	Estado
1	✅ Schema + triggers + audit trail	—	✅ HECHO
2	✅ Probado: DELETE/UPDATE bloqueados, audit_log funciona	—	✅ HECHO
3	✅ **Supabase** — proyecto creado + migrations ejecutadas	5 min	✅ HECHO
4	✅ **Dual-write** — `database.py` SQLite + Supabase REST API	Media	✅ HECHO (19/03/2026)
5	☐ **Crear usuario restringido** — `tsc_app` con solo INSERT/SELECT	5 min	Pre-producción
6	☐ **Backups automáticos** — incluidos en Supabase	Gratis	Automático
7	✅ **Añadir columnas** `furnace`, `duration_s`, `total_minutes`, `client_id`, `pieces_id` a tabla `loads`	1 min	`migrations/002_add_loads_columns.sql`

8.1.9 Supabase — Plan de despliegue producción

¿Por qué Supabase y no PostgreSQL local?

	PostgreSQL en Mac	PostgreSQL en MiniPC	Supabase (cloud)
Alberto (Mac, remoto)	✅	Via Tailscale	✅ Web
Darío (Windows, fábrica)	❌	❌ Necesita cliente	✅ **Web, sin instalar nada**
Gerencia TSC	❌	❌	✅ Web desde cualquier lugar
pgaudit	❌ Homebrew	❌ Hay que compilar	✅ **Incluido**
Backups	Manual	Manual	✅ **Automáticos**
Dashboard web (tablas, SQL)	pgAdmin local	❌	✅ **Supabase Studio**
Funciona sin internet	✅	✅	❌ (SQLite sigue de backup)

```
flowchart TD
    A["Operario pulsa START"] --> B["MiniPC (app.py)"]
    B --> C["1. SQLite local\n(instantáneo, siempre)"]
    B --> D["2. Supabase PostgreSQL\n(async, si hay internet)"]

    C --> E["Dashboard :8080\n(MiniPC local)"]
    D --> F["Supabase Studio\n(web, cualquier navegador)"]

    F --> G["Alberto (Mac)"]
    F --> H["Darío (Windows)"]
    F --> I["Gerencia TSC"]
```

Despliegue paso a paso:

```
# 1. Alberto entra en supabase.com → Sign Up (gratis)
# 2. New Project → nombre: "tsc-monitor" → región: eu-west-1 (Frankfurt)
# 3. Copiar la DB password (se genera automáticamente)
# 4. Ir a SQL Editor → New Query → pegar contenido de:
#    - db/init/001_schema.sql (quitar línea CREATE EXTENSION pgaudit)
#    - db/init/002_audit_trail.sql (quitar línea CREATE EXTENSION pgaudit)
#    → Run
# 5. Copiar la connection string:
#    Settings → Database → Connection string → URI
#    postgresql://postgres.xxxx:PASSWORD@aws-0-eu-west-1.pooler.supabase.com:6543/po
# 6. Añadir al .env del MiniPC:
#    SUPABASE_DB_URL=postgresql://postgres.xxxx:PASSWORD@...
# 7. El backend hará dual-write: SQLite (siempre) + Supabase (si hay internet)
```

¿Quién ve qué?

Persona	Accede desde	Cómo	URL
Alberto	Mac (remoto)	Supabase Studio → tablas, SQL, gráficos	`app.supabase.com`
Darío	Windows (fábrica)	Mismo Supabase con su email	`app.supabase.com`
Operario	Monitor planta	Dashboard local `:8080`	`http://100.104.65.25:8080`

> ✅ **¿Si le doy a START/STOP ahora guarda en Supabase?** → **SÍ.** El dual-write está activo desde el 19/03/2026. Al pulsar START/STOP: 1. ✅ SQLite local (instantáneo, siempre funciona) 2. ✅ Supabase PostgreSQL (async, via REST API HTTPS) 3. ✅ Email via Resend (async)
>
> **Nota técnica**: Supabase no tiene IPv4 para conexión PostgreSQL directa. Se usa la **REST API** (PostgREST) via HTTPS que sí funciona. No se necesita Docker en el MiniPC.

8.1.9.1 Login — Autenticación de usuarios

El dashboard requiere autenticación para acceder. Login en `/login` con logo TSC.

Usuario	Rol	Password
`op1`	Operario 1	`12341234`
`op2`	Operario 2	`12341234`
`admin`	Administrador	`12341234`
`dario`	Darío (gerencia)	`12341234`

Páginas protegidas (requieren login): - `/` — Control de hornos (START/STOP/RESET) - `/dashboard` — Dashboard avanzado (gráficas, histórico, consulta inteligente) - `/db` — Visor de base de datos (trazabilidad completa)

8.1.9.2 Dashboard avanzado (`/dashboard`)

Sección	Descripción
Stats strip	Cargas hoy, eventos hoy, lecturas totales, cargas totales
Gráfica barras	Cargas por día (últimos 30 días) — Chart.js
Gráfica líneas	Duración total por día (minutos) — Chart.js
Consulta Inteligente	Input de texto para consultas. Fase 1: búsqueda por keywords. Fase 2: LLM
Histórico de Cargas	Filtros por fecha y horno, tabla con todas las cargas
Últimos Eventos	Tabla auto-refresh cada 10s

8.1.9.2.1 Calendario y cargas por color

El dashboard incluye un **calendario interactivo** con código de colores por carga:

Carga #	Color	CSS Variable
Carga 1	🟢 Verde	`--load1: #22c55e`
Carga 2	🔵 Azul	`--load2: #3b82f6`
Carga 3	🟡 Amarillo	`--load3: #facc15`
Carga 4+	🔴 Rojo	`--load4: #ef4444`

Interacciones: - **Hover** sobre una carga → tooltip con duración (ej: "2h 3m") - **Click** en un día del calendario → despliega tarjeta lateral con cargas del día - **Click** en una carga → **modal de detalle completo** con: tiempo, temperatura media/max/min, cliente, pieza, OT, proceso, lecturas

API endpoints del calendario: - `GET /api/db/calendar-dates?month=3&year=2026` → fechas con cargas - `GET /api/db/loads-by-date?date=2026-03-19` → cargas de un día - `GET /api/db/load/42` → detalle completo de una carga

8.1.9.2.2 Trazabilidad

> 📄 Ver documento completo: **TSC_TRAZABILIDAD.md** — Proceso ARCOR (SURSULF + OXYNIT), normativa ISO/IATF/AMS, industrias aplicables, tipos de piezas trazables.

8.1.9.2.3 Script de limpieza de datos de test

Para el paso a producción, se incluye `cleanup_test_data.py` :

```
# Dry-run (muestra qué se borraría, sin borrar nada)
python3 cleanup_test_data.py

# Ejecución real (requiere confirmación "YES")
python3 cleanup_test_data.py --confirm
```

Borra datos de `loads`, `events`, `readings`, `work_orders` en SQLite y Supabase, resetea auto-incrementos y hace VACUUM.

8.1.9.3 Dual-Write — Arquitectura técnica

```
flowchart TD
    A["Operario pulsa START"] --> B["app.py (Flask, MiniPC)"]
    B --> C["1. SQLite local\n(instantáneo, WAL mode)"]
    B --> D["2. Supabase REST API\n(async thread, HTTPS)"]
    B --> E["3. Resend Email\n(async thread)"]
    C --> F["Dashboard :8080"]
    D --> G["Supabase Studio"]
    D --> H["Grafana"]
```

¿Por qué REST API y no PostgreSQL directo?

	PG directo	**REST API (PostgREST)**
IPv4	❌ Supabase solo IPv6	✅ HTTPS funciona
Docker	Necesario para psycopg2	❌ No necesario
Firewall	Puerto 5432	Puerto 443 (HTTPS)
Complejidad	Alta	Baja
Resiliencia	Si falla = error	Si falla = SQLite sigue

8.1.10 Dashboard vs Grafana — ¿Qué es cada cosa?

> **Explicación simple**: El Dashboard es el **volante del coche** (controlas el horno). Grafana es el **mecánico que mira el motor** (analiza los datos históricos).

```
┌──────────────────────────────────────────────────────────┐
│                      TU SISTEMA TSC                      │
│                                                          │
│   ┌──────────────┐     ESCRIBE      ┌──────────────────┐ │
│   │  Dashboard   │ ────────────────→│  SQLite (local)  │ │
│   │  :8080       │ ────────────────→│  Supabase (cloud)│ │
│   │              │     DUAL-WRITE   └──────────────────┘ │
│   │ START / STOP │                          │            │
│   │ Temp en vivo │                          ▼            │
│   │ Calendario   │                 ┌────────────────────┐ │
│   └──────────────┘                 │  Grafana :3000     │ │
│                                   │ Lee los datos      │ │
│   OPERARIO usa esto                │  y hace gráficas   │ │
│   en PLANTA                        │  bonitas           │ │
│                                    └────────────────────┘ │
│                                     INGENIERO usa esto    │
│                                     para ANÁLISIS         │
└──────────────────────────────────────────────────────────┘
```

Analogía del restaurante:

	Dashboard (:8080)	**Grafana (:3000)**
🍳 **Qué es**	La **cocina** — donde se trabaja	El **libro de contabilidad** — donde se analiza
👷 **Quién lo usa**	El **operario** en planta	El **ingeniero** / gerente
🎯 **Para qué**	**Controlar** el horno en tiempo real	**Analizar** datos históricos
⏰ **Cuándo**	**Ahora mismo** — ¿qué temperatura hay?	**Después** — ¿cómo fue la carga del martes?
📊 **Datos**	Temperatura actual, START/STOP, calendario	Curvas, tendencias, comparaciones, alarmas
✏️ **Acciones**	SÍ — pulsar START, STOP, ver cargas	NO — solo visualiza, no toca nada
🌐 **Acceso**	Solo desde la red local (planta)	Local + Cloud (Supabase + Grafana Cloud)

Ejemplo práctico:

```
LUNES 8:00 — El operario llega a planta
  → Abre Dashboard (:8080) → pulsa START → mete piezas
  → Ve la temperatura subir en tiempo real
  → A las 10:30 la carga termina → pulsa STOP

LUNES 16:00 — Alberto revisa desde casa
  → Abre Grafana (:3000) → ve la curva de temperatura del lunes
  → Compara con la carga del viernes
  → Ve que tardó 5 min más en llegar a 580°C
  → Detecta que el termopar 2 tiene una desviación de 3°C
```

¿Qué pasa cuando haces START/STOP?

```
PULSAR START en Dashboard
        │
        ▼
   ┌─ database.py ─┐
   │               │
   │  1. SQLite    │ ← SIEMPRE escribe (local, instantáneo)
   │     ✅        │
   │               │
   │  2. Supabase  │ ← Intenta escribir (cloud, necesita internet)
   │     ✅ o ⚠️   │   Si falla → SQLite tiene la copia
   │               │
   └───────────────┘
        │
        ▼
   DATO GUARDADO en ambos sitios (dual-write)
```

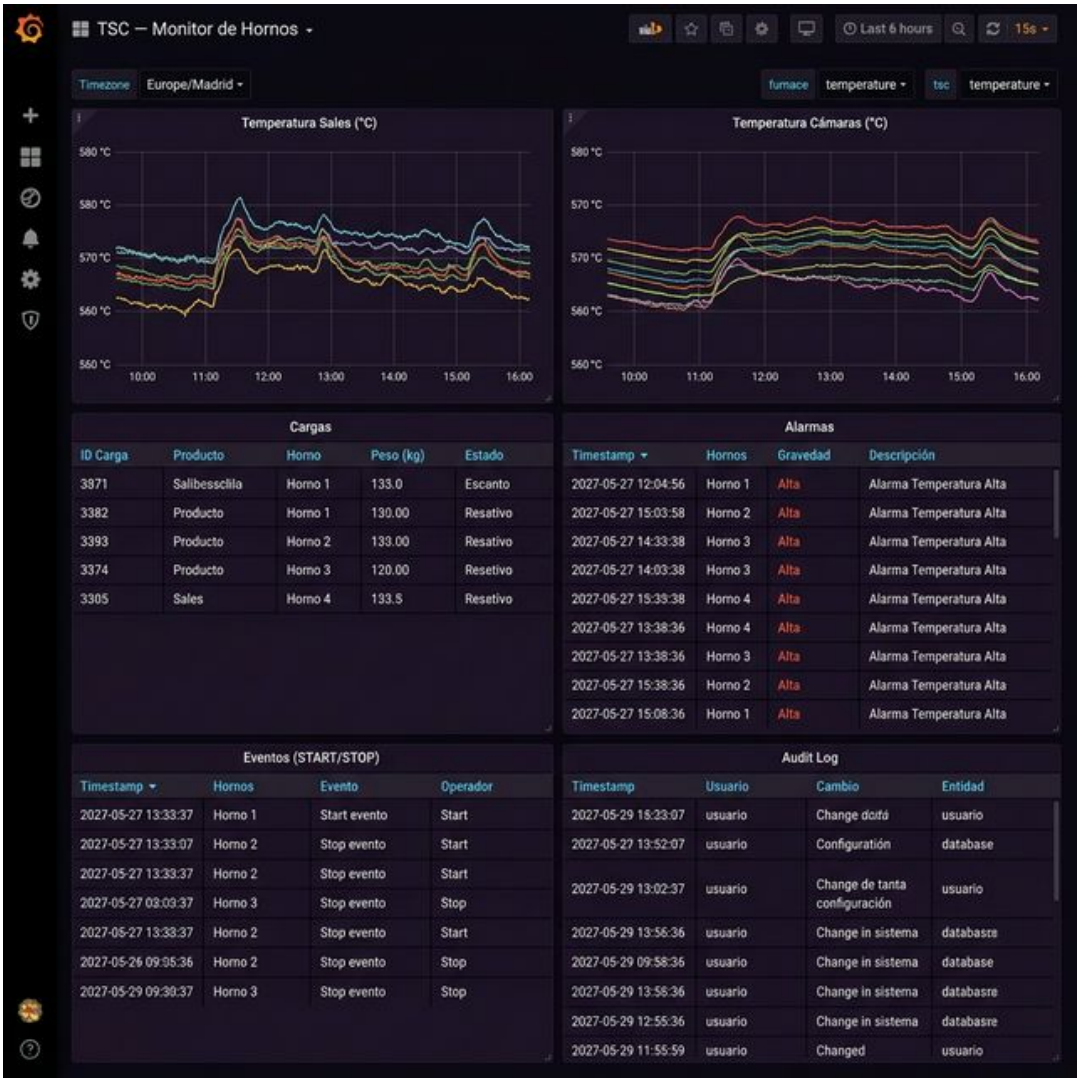
> ⚠️ **IMPORTANTE**: Para que Supabase funcione, el MiniPC necesita las credenciales en `.env`:
>
> ```
> SUPABASE_URL=https://httfohqoikkptywupkrs.supabase.co
> SUPABASE_ANON_KEY=eyJhbG... (tu clave)
> ```
>
> Si `.env` no tiene estas variables, **solo se escribe en SQLite** (que es seguro — no se pierde nada).

8.1.10b Grafana — Dashboard de monitorización avanzada

Grafana corre en `http://localhost:3000` con auto-start vía Homebrew.

Credenciales: `admin` / `12341234`

Dashboard "TSC — Monitor de Hornos" — 6 paneles:



Grafana Dashboard TSC — Monitor de Hornos con curvas de temperatura y tablas de cargas, alarmas, eventos y audit log

Panel	Tipo	Query
Temperatura Sales	Curva temporal	`SELECT timestamp, temp_sales, set_point FROM readings`
Temperatura Cámaras	Curva temporal	`SELECT timestamp, temp_cameras, set_point FROM readings`
Cargas	Tabla	loads + work_orders JOIN
Alarmas	Tabla	alarms ORDER BY timestamp DESC
Eventos (START/STOP)	Tabla	events ORDER BY timestamp DESC
Audit Log	Tabla	audit_log ORDER BY timestamp DESC

```
# Arrancar Grafana
brew services start grafana
# → http://localhost:3000

# Parar
brew services stop grafana

# Re-importar dashboard si se pierde
curl -s -X POST http://admin:12341234@localhost:3000/api/dashboards/db \
  -H "Content-Type: application/json" \
  -d @grafana/tsc-dashboard.json
```

Datasources configurados: | Nombre | Tipo | Host | Base de datos | |—|—|—|—| | TSC PostgreSQL | Local | `localhost:5432` | `tsc_furnace` | | TSC Supabase (Cloud) | Cloud | `db.httfohqoikkptywupkrs.supabase.co:5432` | `postgres` |

8.1.11 Supabase — ✅ Desplegado y verificado

Proyecto: `TSC_OMROM_E5CC` | **Región**: `eu-west-1` (Frankfurt)

Test	Resultado
✅ Schema (5 tablas + 11 índices)	Creado via SQL Editor
✅ Audit trail + triggers inmutables	Creado via SQL Editor
✅ RLS policies	Configuradas (allow all para service role)
✅ INSERT load, reading, event	Verificado via REST API
✅ Audit log automático	3 registros generados automáticamente
✅ DELETE reading bloqueado	Trigger funcionó correctamente

Acceso para el equipo:

Persona	URL	Método
Alberto	`app.supabase.com`	Login con `hola@movildrive.com`
Darío	`app.supabase.com`	Invitar por email desde Organization Settings
Grafana	`localhost:3000`	Datasource “TSC Supabase (Cloud)”

> **Imágenes pendientes** para futuras capturas: - `docs/img/supabase_table_editor.png` — Vista de tablas en Supabase Studio - `docs/img/supabase_sql_editor.png` — SQL Editor ejecutando queries - `docs/img/grafana_temperaturas_real.png` — Curvas con datos reales del Modbus - `docs/img/dashboard_8080.png` — Dashboard del MiniPC en planta

8.2 Inteligencia Artificial — Motor de Consultas LLM ✅

> **Estado**: ✅ **IMPLEMENTADO y en producción** (29/03/2026) **Acceso**: `https://tsc.tail8dce43.ts.net/db` → pestaña **LLM**

Arquitectura Híbrida

```
flowchart TD
    A["Usuario escribe pregunta\nen lenguaje natural"] --> B{"¿Tema TSC?"}
    B -- "NO" --> C["Respuesta graciosa\nde rechazo"]
    B -- "SÍ" --> D["Motor Local\n(regex + SQL)"]
    D --> E{"¿Patrón reconocido?"}
    E -- "SÍ" --> F["⚡ SQL instantáneo"]
    E -- "NO" --> G["🤖 Ollama phi3:mini\n(timeout 15s)"]
    G --> H{"¿SQL válido?"}
    H -- "SÍ" --> F
    H -- "NO" --> I["Error: reformular"]
    F --> J["Ejecutar SELECT\nen SQLite"]
    J --> K["Mostrar tabla\nde resultados"]
    J --> L["Guardar en\nllm_searches"]
```

Motor Local — Sinónimos soportados

Concepto	Palabras que entiende
Temperatura	temperatura, temperature, temp, grados, °C, térmica, calor
Media	media, promedio, average, avg, mean
Máxima	máxima, max, pico, mayor, alta, más alta
Mínima	mínima, min, menor, baja, más baja
Cargas	carga, cargas, load, loads, batch, lote
OT	ot, ots, orden de trabajo, órdenes, numero de ot, work order
Eventos	evento, alarma, start, stop, arranque, parada, incidencia
Duración	duración, duration, cuánto duró, cuánto tardó, tiempo de
Recientes	última, ultimas, últimos, reciente, hoy, last, recent
Conteo	cuántas, total de, count, se hicieron, hubo, hay

Parseo de fechas y horas

El motor entiende: - `25/03/2026` , `25/03/26` , `25/3/2026` → `2026-03-25` - `de 9 a 14h` , `de 9hrs a 14hrs` , `de 09:00 a 14:00` → rango horario

Ejemplos de queries

Pregunta	Motor	SQL generado
`Temperatura media del 25/03/2026 de 9 a 14h`	⚡ local	`SELECT AVG(temp_sales)... WHERE timestamp BETWEEN...`
`Cuantas cargas el 24/03/2026?`	⚡ local	`SELECT COUNT(*) FROM loads WHERE date = '2026-03-24'`
`OTs del 24/03/2026`	⚡ local	`SELECT ot_number, name... FROM loads WHERE date = ...`
`Últimas 10 cargas`	⚡ local	`SELECT ... FROM loads ORDER BY start_time DESC LIMIT 10`
`Eventos del 25/03/2026`	⚡ local	`SELECT ... FROM events WHERE date(timestamp) = ...`
`¿Quién es Messi?`	🚫	🏭 `Mi mundo empieza y acaba en el horno...`

Filtro TSC — Solo preguntas del taller

Si la pregunta no contiene keywords de TSC (temperatura, carga, OT, horno, etc.), el sistema responde con humor:

- 🔥 *"Yo solo sé de hornos, temperaturas y cargas. ¡Pregúntame algo del taller!"*
- 🤷 *"Ni idea… Yo soy el cerebro del horno, no Google."*
- 🏭 *"Mi mundo empieza y acaba en el horno. Pregúntame por temperaturas, cargas u OTs."*

Logging — Tabla `llm_searches`

Todas las búsquedas se guardan en **SQLite local + Supabase** para análisis de uso:

Campo	Tipo	Descripción
`timestamp`	DATETIME	Momento de la consulta
`question`	TEXT	Pregunta del usuario
`sql_generated`	TEXT	SQL generado (si hubo)
`engine`	TEXT	`local` o `ollama`
`result_count`	INTEGER	Número de resultados
`error`	TEXT	Mensaje de error (si falló)
`date`	DATE	Fecha para indexación

Ollama — Fallback con phi3:mini

Parámetro	Valor
Modelo	`phi3:mini` (3.8B params)
Timeout	15 segundos
temperature	0 (determinista)
num_predict	200 tokens max
Endpoint	`http://localhost:11434/api/generate`

```
# Ya instalado en MiniPC. Para verificar:
ssh tsc-tailscale 'ollama list'
```

8.2.1 Roadmap IA — Próximas fases

Fase	Feature	Estado
✅	Motor local pattern-matching	**Producción**
✅	Fallback Ollama phi3:mini	**Producción**
✅	Filtro TSC-only con humor	**Producción**
✅	Logging en llm_searches	**Producción**
⬜	Voz → OT (Whisper + LLM)	Fase 2
⬜	Detección anomalías IA	Fase 2
⬜	Informes semanales auto	Fase 3

9. Hasta dónde podemos llegar

FASE 1 (actual) ✅	FASE 2 ⬜	FASE 3 (futuro) ⬜
Lectura Modbus	PostgreSQL cloud	Multi-planta
Dashboard básico	Voz → OT	Gemelo digital 3D
START/STOP timer	Consultas naturales	Mantenimiento predictivo
Email Resend	Anomalías IA	Integración ERP
SQLite local	Informes automáticos	Certificación ISO
Simulador	Dashboard v2	App móvil nativa

9.1 ¿Qué es un Gemelo Digital?

Un **gemelo digital** (digital twin) es una réplica virtual 3D del horno de TSC que muestra datos reales en tiempo real. En nuestro caso:

Aspecto	Cómo funciona en TSC
Qué se ve	Modelo 3D del horno SURSULF con zonas de color según temperatura (azul→rojo)
Datos en vivo	Las temperaturas reales de sales y cámaras se mapean visualmente sobre el modelo
Para qué sirve	El operario o el gerente ve de un vistazo dónde hay puntos calientes, anomalías o fallos
Mantenimiento	Predecir desgaste de resistencias visualizando patrones térmicos en el modelo
Auditoría	Reproducir cargas pasadas en el modelo 3D para analizar incidencias
Tecnología	Three.js (motor 3D web) + WebSocket (datos en vivo)
Ejemplo real	Como un Google Maps pero de tu horno — zoom, rotar, ver temperaturas por zona

> En TSC serviría para que el gerente desde su oficina vea el estado exacto de cada horno sin ir a planta, y para detectar problemas antes de que ocurran.

9.2 Fase 3 — Visión completa

Feature	Descripción	Tecnología
Multi-planta	Monitorizar varias fábricas desde un solo panel	Supabase multi-tenant
Gemelo digital	Modelo 3D del horno con datos en tiempo real	Three.js + WebSocket
Mantenimiento predictivo	IA predice cuándo fallará una resistencia antes de que pase	TimescaleDB + ML
Integración ERP	OTs se crean automáticamente desde el ERP del cliente	API REST bidireccional
Certificación ISO	Audit trail inmutable con firma digital para normativa	pgaudit + blockchain hash
App móvil nativa	El operario ve temperaturas en su móvil	React Native / Flutter

10. Conectividad y Redes

10.1 Conexiones del equipo TSC

Nombre	Tipo	IP	Interfaz	Ubicación
Sercomm1D20	WiFi	`192.168.0.17`	`wlp1s0`	Casa de Alberto
Livebox6-9D67	WiFi	*(DHCP)*	`wlp1s0`	Fábrica TSC
HONOR	WiFi (hotspot)	—	—	Móvil (backup)
tailscale0	VPN	`100.104.65.25`	`tailscale0`	Acceso remoto global
NordVPN	VPN	*(dinámica)*	`nordlynx` / `tun0`	Privacidad / bypass

10.2 Credenciales WiFi

Red	SSID	Password
Casa Alberto	`Sercomm1D20`	*(ya guardada en equipo)*
Fábrica TSC	`Livebox6-9D67`	`tcQ5x3eYoaed`

10.3 SSH — Acceso remoto

Configuración actual (`~/.ssh/config` en tu Mac):

```
Host tsc
    HostName 100.104.65.25
    User tsc
```

Para conectarte: `ssh tsc`

10.4 Tailscale — VPN

Dispositivo	IP Local	IP Tailscale	Rol
PC Planta	192.168.1.X	100.104.65.25	Backend + Dashboard
Mac Alberto	—	100.x.x.x	Desarrollo remoto
Monitor	192.168.1.X	—	Solo display

> ⚠️ **Actualizar IPs** después de la instalación in situ.

11. Troubleshooting — Cheat Sheet

Acción	Comando
Ver redes WiFi	`nmcli dev wifi list`
Escanear redes	`nmcli dev wifi rescan`
Conectar WiFi	`nmcli connection up "Livebox6-9D67"`
Desconectar WiFi	`nmcli connection down "Sercomm1D20"`
Ver conexiones activas	`nmcli connection show --active`
Ver IP WiFi	`ip -4 addr show wlp1s0`
Ver IP Tailscale	`tailscale ip -4`
Estado general	`nmcli dev status`
SSH al equipo	`ssh tsc`
Reiniciar red	`sudo systemctl restart NetworkManager`
Reiniciar Tailscale	`sudo systemctl restart tailscaled`
Ver todas las IPs	`ip addr show | grep "inet "`

> 📖 Ver troubleshooting completo paso a paso en ARCHITECTURE.md

🔐 Credenciales y Accesos

> ⚠️ **NO subir este archivo a repositorios públicos.** Las credenciales sensibles también están en `.env`.

Servicios

Servicio	URL	Usuario
Grafana (local)	`http://localhost:3000`	`admin`
PostgreSQL (local)	`localhost:5432 / tsc_furnace`	`alberto`
Supabase Studio	`https://supabase.com/dashboard`	`hola@movildrive.com`
Supabase DB (direct)	`db.httfohqoikkptywupkrs.supabase.co:5432`	`postgres`
Dashboard TSC	`http://100.104.65.25:8080`	—
SSH MiniPC	`ssh tsc ( 100.104.65.25 )`	`tsc`

Supabase — Proyecto TSC_OMROM_E5CC

```
PROJECT_URL=https://httfohqoikkptywupkrs.supabase.co
ANON_KEY=sb_publishable_RpXnNg8xARuRnsHLve6aUQ_gMJxc1sf
DB_URL=postgresql://postgres:Trabajar81!@db.httfohqoikkptywupkrs.supabase.co:5432/po
```

WiFi

Red	Password
Livebox6-9D67 (fábrica)	`tcQ5x3eYoaed`
Sercomm1D20 (casa Alberto)	*(guardada en equipo)*

API Keys

Servicio	Key
Resend (email)	`re_PJ4SJgLF_KJXVutpAfyoEWsjdpB8ncJfg`
GitHub	`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`

📄 Licencia

© NexusFinLabs 2026 — Todos los derechos reservados.